

Chapter 00 — Overview & Architecture

Start here. This chapter is the map; Chapters 01–07 are the detail. Full narrative version: `project-brief/manual.md`.

1. What CodeAstra is

An **AI-powered codebase-intelligence agent**: give it a GitHub repo URL, get back a structured, human-readable understanding of the project — so a developer can onboard in minutes, not days.

Three mandatory analyses:

Module	Feature	Chapter
M1	Folder Structure Analysis — purpose + category of each folder	01
M2	Entry Point Detection — start file + execution flow	02
M3	Dependency Mapping — import graph + ASCII tree	03

2. Tech stack

Backend — Node.js, Express 5, TypeScript, MongoDB (Mongoose), LangGraph + LangChain, **NVIDIA Nemotron** LLMs (OpenAI-compatible endpoint `integrate.api.nvidia.com/v1`), Octokit, Zod.

Frontend — React 19, TypeScript, Vite, TailwindCSS 3, framer-motion, React Router v7, ReactFlow + dagre.

Doc accuracy: older `.docs` files mention Groq/Llama and Gemini — both are superseded. Current code (`backend/src/ai/model.ts`) is Nemotron: `nemotron-3-nano-30b-a3b` (M1) and `nemotron-3-super-120b-a12b` (M2, M3), all `temperature: 0`.

3. Visual architecture

3.1 Full system architecture

client

User

frontend

Frontend (React)

Auth UI

Login / register

Repo input

GitHub URL form

Results dashboard

M1 · M2 · M3 · Bonus

cached

backend

Backend API (Node.js / Express)

Auth service

JWT · bcrypt

Repo controller

Validate URL · queue

Cache service

Redis / MongoDB

AI layer

AI Layer — LangGraph workflow

Data fetch

GitHub API calls
rate-limit guard

Parser

AST · imports
chunk + embed

Analysis

M1 · M2 · M3
B1 · B2 · B3

Structured output generator

Claude API · prompt chains · JSON schema
citations + M1/M2/M3 sections

Vector store

embeddings · similarity search

LangGraph state

workflow graph · retry logic

result

data

GitHub API

repo tree · file contents
OAuth token (5000 req/hr)

MongoDB

users · analysis results
repo cache by URL hash

Redis

job queue
short-lived cache

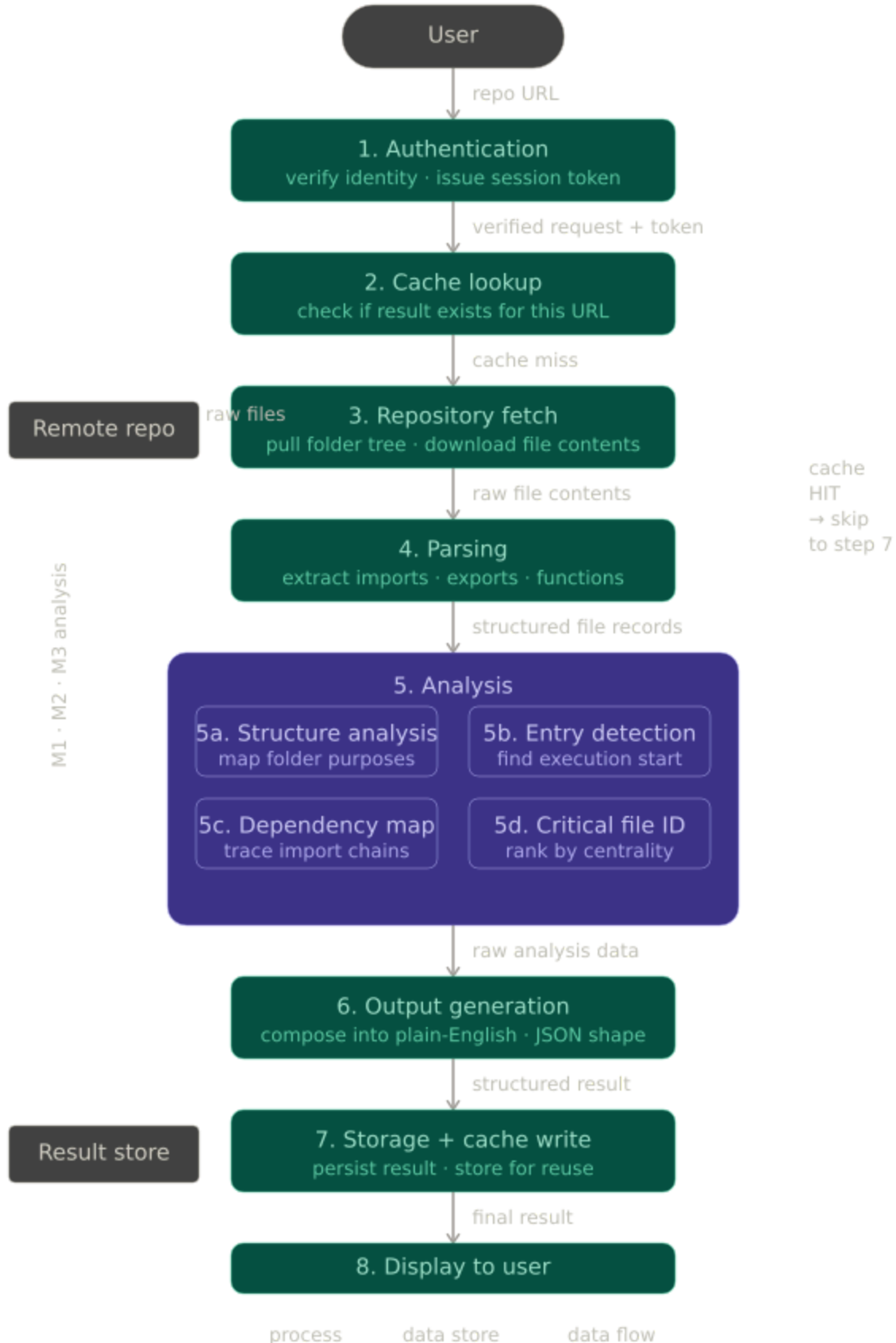
Legend

→ primary flow

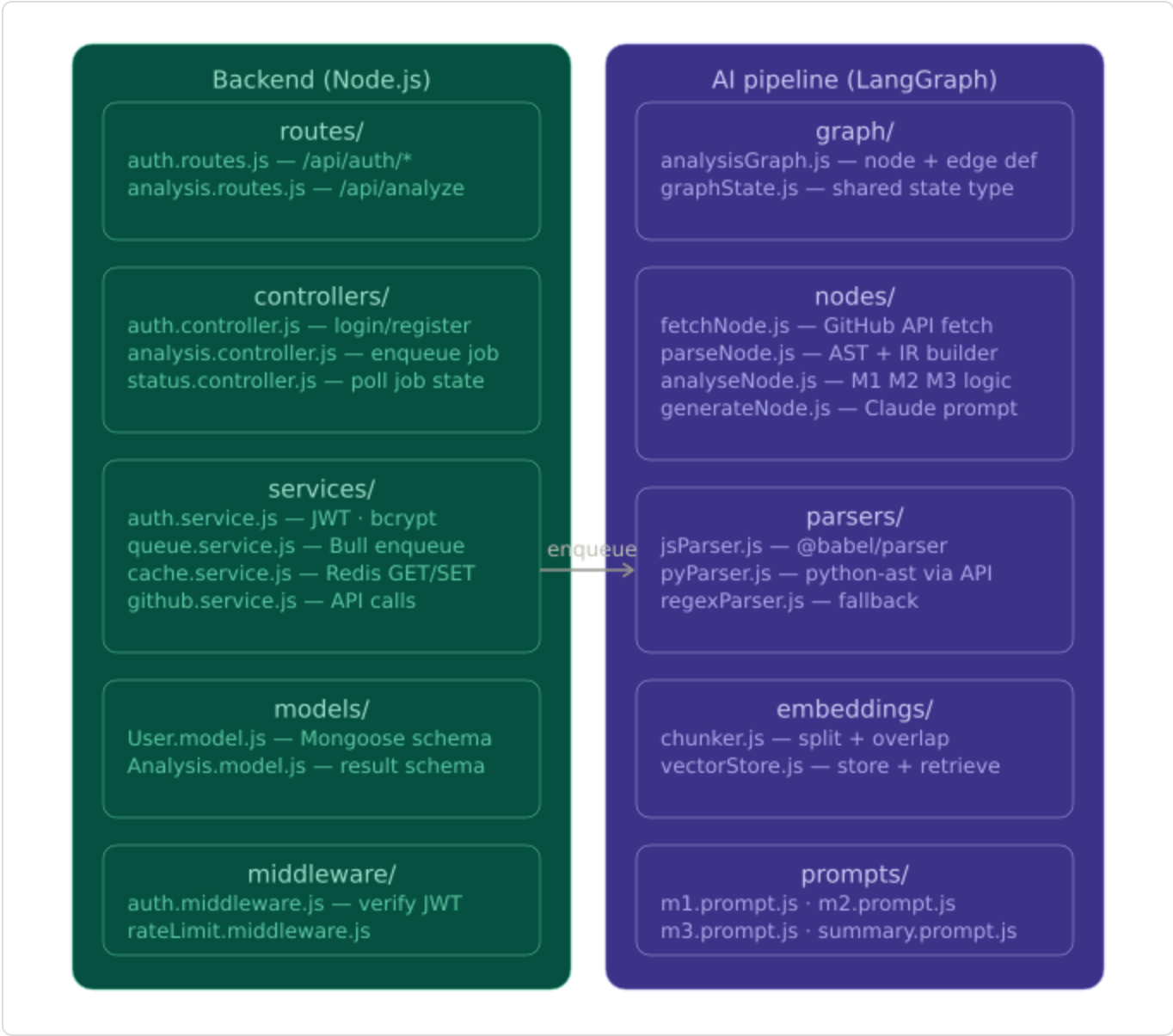
data / cache access ■ neutral / infra

M1 = folder analysis · M2 = entry point · M3 = dependency map · B1/B2/B3 = bonus

3.2 Data flow



3.3 Low-level module structure



4. End-to-end flow (one picture)

```

GitHub URL
|
▼ POST /api/analysis (Chapter 06)
Controller —SHA256—► Mongo cache — hit —► return cached
| miss
▼
AI Service orchestrator (Chapter 04)
├─ GitHub Service: tree → source files (60-cap, 5-batch) (Chapter 05)
├─ Parser Service: regex IR (imports/exports/functions) (Chapter 05)
├─ resolve entry contents (top 3) (Chapter 02)
▼
LangGraph: START → [ M1 || M2 || M3 ] → Combine → END (Chapters 01/02/03/04)
|
└─ or AST/regex fallback on LLM failure
▼
IAnalysisResult → save to Mongo → JSON to client
|
▼
React SPA renders dashboard / tree / graph (Chapter 07)

```

5. Run it

```

# Backend
cd backend && npm install
# .env: PORT=4000 MONGO_URI=... GITHUB_TOKEN=ghp_... NVIDIA_API_KEY=nvapi_...
npm run dev # tsx watch

# Frontend
cd frontend && npm install
npm run dev # Vite

```

6. Where to go next

- **Features:** Chapter 01 (M1), 02 (M2), 03 (M3).
- **Engine:** Chapter 04 (LangGraph pipeline).
- **Ingestion:** Chapter 05 (GitHub + parser).
- **API/DB:** Chapter 06.
- **UI:** Chapter 07.
- **Current issues & plan:** `project-brief/plan.md` . **Running history:** `project-brief/context-log.md` .